



CORRIGINDO UMA CALCULADORA COM INTERFACE GRÁFICA EM PYTHON

GUIA VISUAL PASSO A PASSO COM **TKINTER SIMPLÉS**

1 O QUE ESTE PROGRAMA FAZ?

Esta calculadora permite que o usuário:

- ✓ Digite dois números
- ✓ Escolha uma operação matemática
- ✓ Veja o resultado na tela

Operações disponíveis:

- 1 Adição
- 2 Subtração
- 3 Multiplicação
- 4 Divisão
- 5 Potência



- i** **Nosso objetivo:**
Corrigir todos os erros do código para que a calculadora funcione corretamente!

2 OS ERROS ENCONTRADOS

Veja todos os problemas que existem no código:

- ✗ 1 Função `adicao()` sem `:`
- ✗ 2 Função `divisao()` com parâmetro faltando
- ✗ 3 Função `potencia()` com parâmetro faltando
- ✗ 4 Função `calcular()` sem a palavra `def`
- ✗ 5 Conversão do primeiro número incorreta
- ✗ 6 Variável `numero2` não criada
- ✗ 7 Estrutura `if` ausente
- ✗ 8 Código da multiplicação incorreto
- ✗ 9 Função `subtracao()` sem implementação
- ✗ 10 Potência não executada corretamente
- ✗ 11 Mensagem final sem variável

- !** Esses erros impedem o programa de executar ou fazem com que o resultado não seja correto.

3 Corrigir a função `adicao()`

Como estava (errado):

```
def adicao(a, b):
```

Como corrigir:

```
def adicao(a, b):  
    return a + b
```

- i** **O que aprendemos?**
Toda função em Python precisa terminar com dois pontos (`:`).

4 Corrigir a função `divisao()`

Como estava (errado):

```
def divisao(a, b):  
    if b == 0:  
        return "Erro: divisão por zero!"  
    return a / b
```

Como corrigir:

```
def divisao(a, b):  
    if b == 0:  
        return "Erro: divisão por zero!"  
    return a / b
```

- i** **O que aprendemos?**
Os parâmetros são os valores que a função recebe.

5 Corrigir a função `potencia()`

Como estava (errado):

```
def potencia(a,):  
    return a ** b
```

Como corrigir:

```
def potencia(a, b):  
    return a ** b
```

- i** **O que aprendemos?**
Todos os parâmetros utilizados dentro da função precisam ser declarados.

6 Corrigir a função `calcular()`

Como estava (errado):

```
calcular(operacao):  
    try:  
        ...
```

Como corrigir:

```
def calcular(operacao):  
    try:  
        ...
```

- i** **O que aprendemos?**
A palavra `def` é utilizada para criar funções.

7 Corrigir o primeiro número

Como estava (errado):

```
num1 = (entry_num1.get())
```

Como corrigir:

```
numero1 = float(entry_num1.get())
```

- i** **O que aprendemos?**
O método `get()` retorna texto. Usamos `float()` para converter para número decimal.

8 Corrigir o segundo número

Como estava (errado):

```
f = float(entry_num2.get())
```

Como corrigir:

```
numero2 = float(entry_num2.get())
```

- i** **O que aprendemos?**
Toda informação precisa ser armazenada em uma variável.

12 Corrigir a potência

Como estava (errado):

```
resultado = (numero1, numero2)
```

Como corrigir:

```
resultado = potencia(numero1, numero2)
```

- i** **O que aprendemos?**
Para executar uma função precisamos chamá-la.

13 Corrigir a mensagem final

Como estava (errado):

```
else:  
    "Operação não encontrada."
```

Como corrigir:

```
else:  
    resultado = "Operação não encontrada."
```

- i** **O que aprendemos?**
Mensagens finais devem ser armazenadas em uma variável.

9 Corrigir a estrutura `if`

Como estava (errado):

```
operacao == "1":  
    resultado = adicao(  
        numero1, numero2)
```

Como corrigir:

```
if operacao == "1":  
    resultado = adicao(  
        numero1, numero2)
```

- i** **O que aprendemos?**
O comando `if` inicia uma estrutura de decisão.

10 Corrigir a multiplicação

Como estava (errado):

```
elif operacao == "1":  
    resultado = multiplicacao(  
        numero1, numero2)
```

Como corrigir:

```
elif operacao == "3":  
    resultado = multiplicacao(  
        numero1, numero2)
```

- i** **O que aprendemos?**
Cada botão possui um código associado à sua operação.

11 Corrigir a subtração

Como estava (errado):

```
def subtracao(a, b):  
    pass
```

Como corrigir:

```
def subtracao(a, b):  
    return a - b
```

- i** **O que aprendemos?**
A instrução `return` devolve o resultado da função.

14 ENTENDENDO A INTERFACE (TKINTER)



Janela Principal
`janela = tk.Tk()`
Cria a janela principal da aplicação.



Campos de Entrada
`entry_num1`
`entry_num2`
Permitem que o usuário digite os números.



Botões de Operação (Buttons)
Ao clicar em um botão, a função `calcular()` é executada com o código da operação.



Resultado
(`label_resultado`)
Mostra o resultado da operação na tela.

15 RESUMO DO QUE APRENDEMOS

- ✓ Criamos e corrigimos funções (`def`)
- ✓ Trabalhamos com parâmetros (`a, b`)
- ✓ Usamos `return` para retornar resultados
- ✓ Convertimos texto para número com `float()`
- ✓ Utilizamos estruturas condicionais (`if / elif / else`)
- ✓ Chamamos funções corretamente
- ✓ Manipulamos variáveis
- ✓ Construímos uma interface gráfica com Tkinter
- ✓ Utilizamos `Entry`, `Button` e `Label`.

OBJETIVO ALCANÇADO!
A calculadora está funcionando corretamente!
Parabéns! Continue praticando! 🏆